VT PROJECT

Observability for AI Agents

Systematic Analysis and Evaluation of Open-Source Tooling

AUTHOR
Daniela Otelea

SUPERVISOR
Dr. Leonardo Militano

PROGRAMME
MSE — Computer Science

PRESENTATION OVERVIEW

Five movements, thirteen slides

01	**The problem** What agents are and why they break traditional monitoring	slides 3–5
02	**The standard** OpenTelemetry as the portability layer	slide 6
03	**What I built & how I evaluated** Prototype, methodology and tool selection	slides 7–9
04	**Results** Per-tool findings and direct comparison	slides 10–11
05	**Best practices, limitations, conclusions** What I learned and where the field still falls short	slides 12–13

THE PROBLEM · 01

What is an agent?

An LLM that **perceives** its environment, **decides** on actions, **executes** tools, and **loops** until a goal is met.

WHAT MAKES THEM HARD TO OBSERVE

Stochastic, non-deterministic outputs

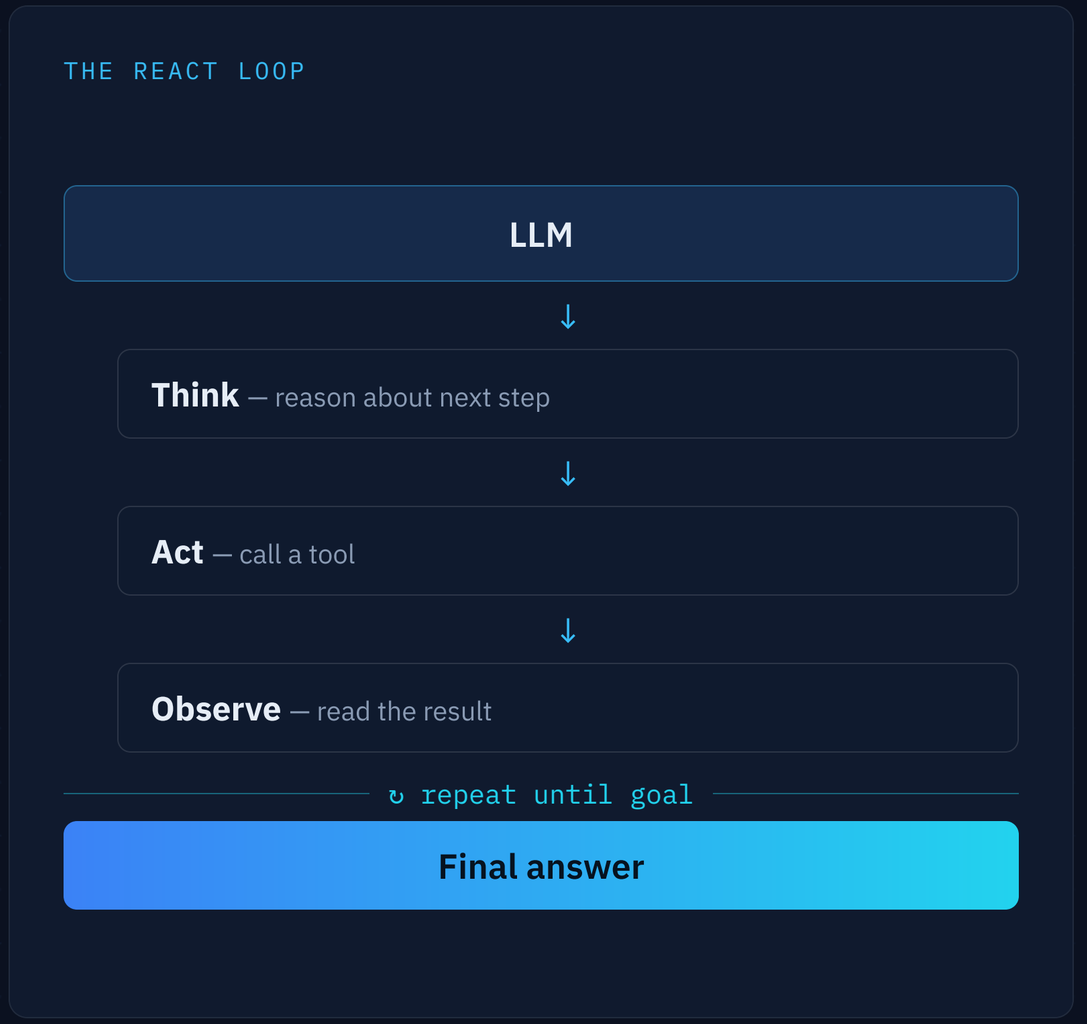
Multi-step reasoning chains

Tool side-effects on the outside world

Long-running or autonomous execution

COORDINATION PATTERNS STUDIED

Single agent (ReAct) | Orchestrator–sub-agent | Evaluator feedback loop



THE PROBLEM · 02

Why traditional observability is not enough

TRADITIONAL REQUEST

request → → 200 OK

Fixed input → predictable output

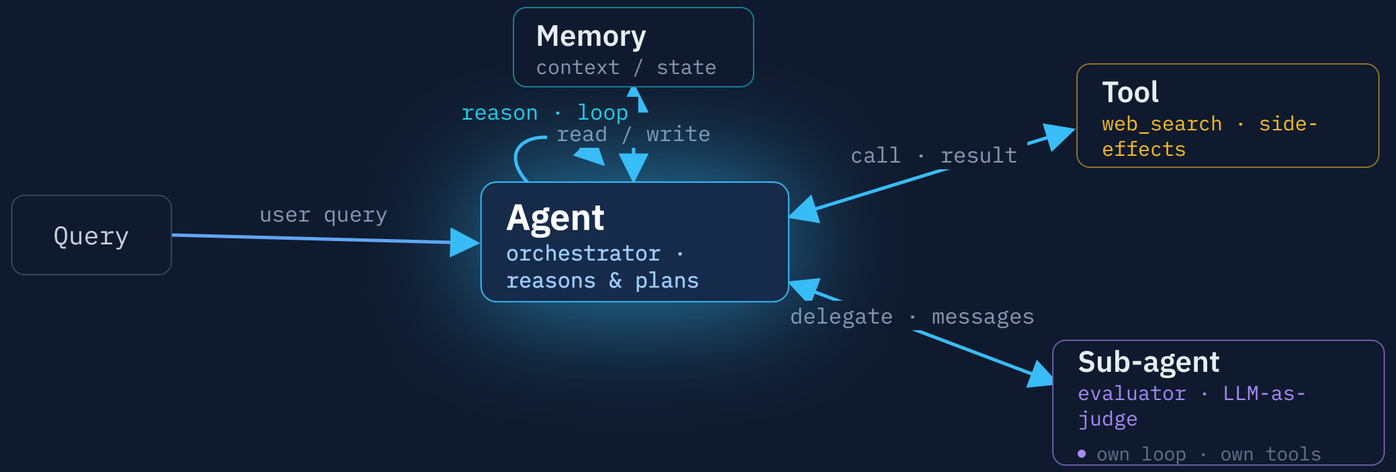
Short-lived request

Clear pass / fail

One span. One answer.

ONE AGENTIC RUN

every edge is where monitoring breaks



Non-deterministic loops, tool side-effects, cross-process messages and memory that grows each turn — "did it succeed?" is no longer binary.

> *"LLM Observability and Explainable AI Form Critical Trust Layers for Scaling GenAI Initiatives"*
>
> — GARTNER · analysts now treat this as a blocking risk for enterprise GenAI

THE PROBLEM · 03 · EVALUATION FRAMEWORK

What comprehensive observability should include

Six dimensions from my own requirements analysis — this framework scored every tool in the evaluation.

01

Reasoning traceability

Chain-of-Thought (CoT) steps, context-window evolution, reasoning depth

02

Tool usage tracking

Input, output and latency per tool call

03

Multi-agent coordination

Inter-agent messages, shared state, throughput

04

Performance metrics

Latency P95 / P99, token usage, cost per role

05

Quality metrics

Faithfulness score, completeness, hallucination rate

06

Safety & governance

PII detection, loop detection, HITL escalation, guardrails

THE STANDARD

OpenTelemetry and OpenInference

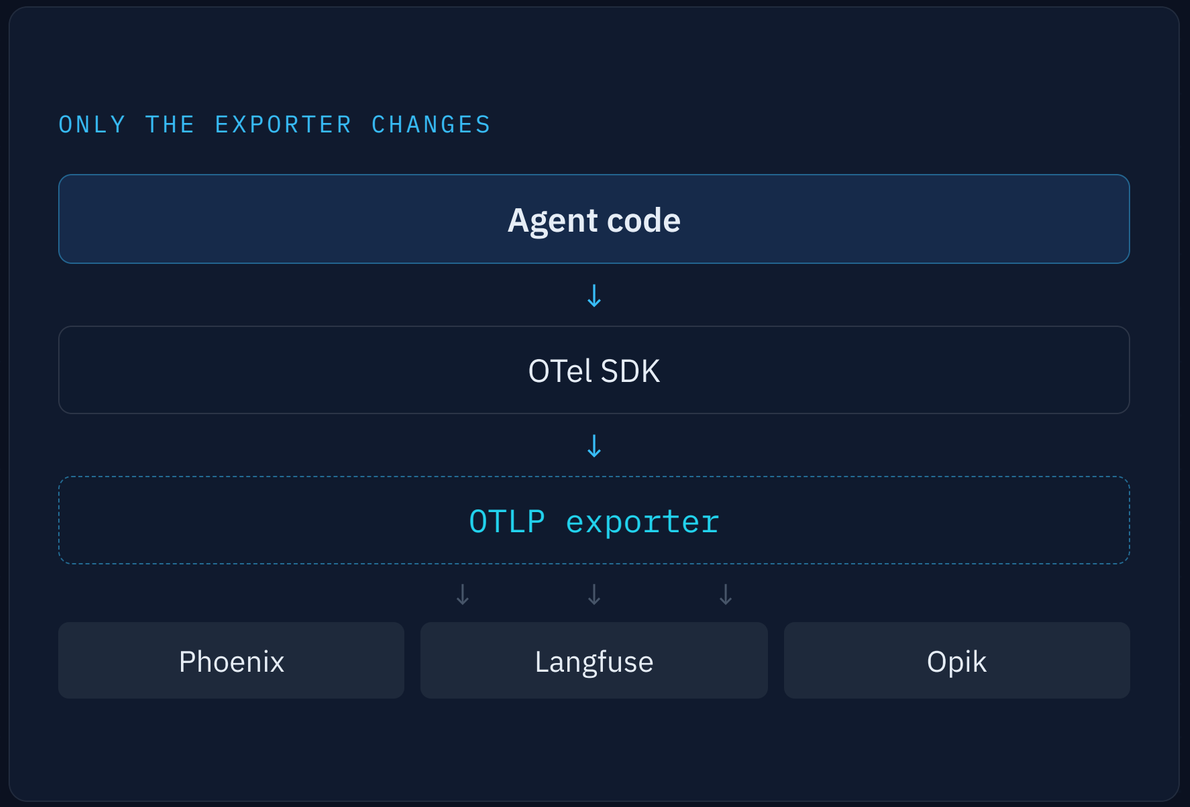
Vendor-neutral instrumentation

Same agent code targets any backend by swapping one exporter — the evaluation is fair because instrumentation is identical.

```
llm.token_count.*
openinference.span.kind
# + openinference-instrumentation-langchain
```

W3C traceparent
links spans across service boundaries

W3C baggage
carries session.id across 3 processes



KEY FINDING

Arize Phoenix is built natively on the OpenInference schema. Langfuse and Opik feature built-in OTLP ingestion pipelines that automatically intercept and translate OpenInference telemetry into their proprietary backends, providing plug-and-play portability without vendor lock-in.

WHAT I BUILT · 63 TESTS PASSING

Two progressively complex agent workloads

ROUND 1 · SIMPLE

SimpleAgent

↓ ReAct loop

add
multiply
divide

Deterministic — verifies span capture & cost tracking

ROUND 2 · V1 IN-PROCESS

Orchestrator

↓

Researcher · `web_search`

Evaluator · `LLM-as-judge`

Retry on low confidence · HITL · LangGraph

ROUND 2 · V2 A2A DISTRIBUTED

Orchestrator :8002

↓ A2A JSON-RPC + W3C headers

Researcher :8011

Evaluator :8012

traceparent + baggage span & session correlation across 3 processes + Agent 2 Agent

All three workloads → pluggable OTel exporter ⟶ Phoenix Langfuse Opik

HOW I EVALUATED · METHODOLOGY

Structured and reproducible

THREE-PILLAR FRAMEWORK

P1 Integration
SDK quality, OTel compliance, auto-instrumentation, setup with Docker

P2 Capability
Metric coverage against the 6-dimension framework

P3 Operability
Deployment model, UI, query capability, APIs, performance overhead

FULL GRID · 2 ROUNDS × 3 TOOLS

	Phoenix	Langfuse	Opik
R1	✓	✓	✓
R2	✓	✓	✓

Round 1 — 5 arithmetic queries, single → 3 chained calls · ≥5 sessions/tool

Round 2 — 5 research queries: focused, comparative, multi-part · ≥5 sessions/tool

`experiments/*/run_experiment.py → structured JSON per session`

HOW I EVALUATED · TOOL SELECTION

Three distinct positions in the landscape

Arize Phoenix

OTel-native · developer-focused

Native OTLP ingestion, local Docker, OpenInference semantic conventions.

1 container

Langfuse

evaluation-heavy · tracing + scoring

Strong LLM-native eval features, ClickHouse backend, widely adopted.

5+ containers

Comet Opik

full-stack ML + LLM

Fastest ingestion in initial comparison, experiment-tracking angle.

8 containers

All three **open-source & self-hostable** (Docker Compose) — no vendor lock-in during evaluation.

EXCLUDED LangSmith (not open-source) · LangWatch (less mature at evaluation time)

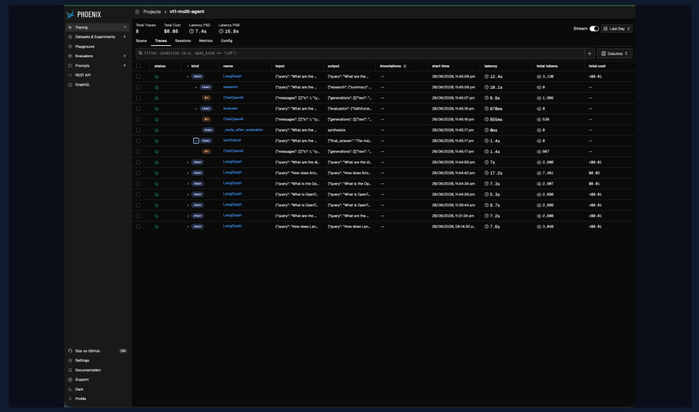
RESULTS · PER-TOOL FINDINGS

A distinct profile for each tool

Arize Phoenix

1 trace

Lowest friction; only tool using standard OTLP and W3C headers end-to-end.



Single container · auto-instrumentation, no per-call config

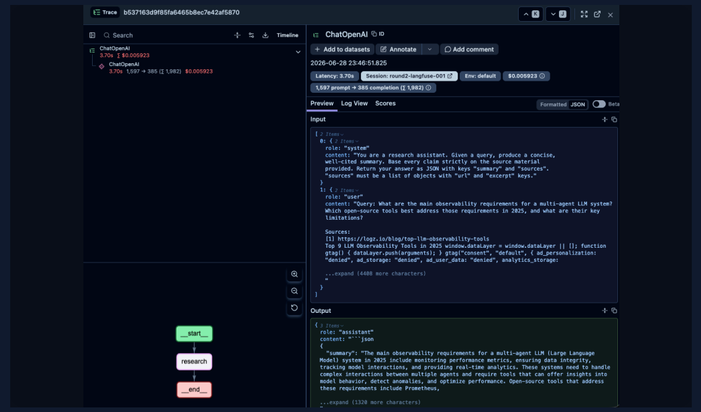
Sub-agent spans become children via W3C `traceparent`

✕ `langgraph_node` metadata not captured

Langfuse

19 /sess

Strongest evaluation UI; highest setup complexity; trace count grows with retries.



5+ containers · API key + project required

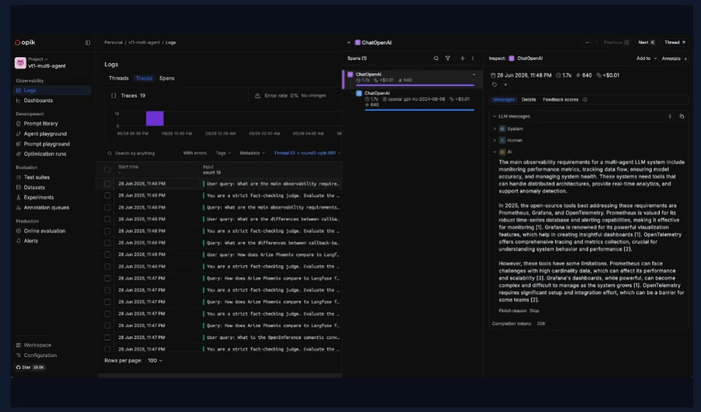
Per-call callback injection — miss it, no trace

✓ Full scoring UI, built-in PII masking

Comet Opik

5 /sess

Zero-friction start; only tool that captures `langgraph_node` automatically.



8 containers but no account — two env vars & `./opik.sh`

✓ LangGraph node / path / triggers metadata

Thread grouping via `thread_id`

RESULTS · HEAD-TO-HEAD

No single winner across all dimensions

✓ Native ~ Partial × None

Dimension	Phoenix	Langfuse	Opik
Reasoning + tool-usage tracking	✓	✓	✓
Multi-agent trace structure	✓ 1/query	~ 19/sess	~ 5/sess
Quality metrics (faithfulness)	×	×	×
LangGraph node metadata	×	×	✓
OTLP wire compliance · auto-instrumentation	✓	×	×
Setup friction	1 container	5+	8
Evaluation / scoring UI · PII masking	×	✓	~
Cross-process span correlation (A2A v2)	✓ W3C	~ proprietary	~ proprietary

DEV / RESEARCH
Phoenix — lowest friction, best A2A correlation

PRODUCTION + COMPLIANCE
Langfuse — scoring UI, PII masking, isolation

LANGGRAPH / ML TRACKING
Opik — node metadata, fastest ingest

BEST PRACTICES & LIMITATIONS

Practical guidelines — and where the field falls short

BEST PRACTICES

B1 Emit **both token namespaces** — `gen_ai.usage.*` + `llm.token_count.*`

B2 Set **OTel Resource attributes** explicitly — service name, version, environment

B3 Promote **session.id into W3C Baggage** for distributed agents

B4 Per-call session callbacks for **Langfuse and Opik** — every `agent.run()`

B5 Use a **fixed query set** — separates tool capability from lucky outputs

B6 Design a **backend-independent TraceEvent audit trail**

B7 Design for **multi-turn sessions** from the start — session.id on every span

LIMITATIONS OF CURRENT TOOLS & STANDARDS

- **Inconsistent semantic conventions** — `gen_ai.*` not natively consumed by any of the three
- **No standardised agent identifiers** — no `agent.id` / `agent.role` in OTel semconv
- **Evaluation metric gap** — in-agent faithfulness scores not ingested as first-class records
- **Trace/metric explosion** — 19 traces/session at 3 agents + 2 retries; no role-level aggregation
- **Data redaction** — prompts & outputs land verbatim; only Langfuse masks PII
- **A2A / MCP not yet supported** — correlation works, but no A2A-aware UI or MCP-native span kinds

CONCLUSIONS

Reproducible evaluation is possible today — the field is still maturing

PRIMARY CONTRIBUTION

A **2-round methodology** that exposes tool limitations only visible at multi-agent complexity — all three scored almost identically on Round 1; the Round 1 → Round 2 delta is where differentiation appeared.

SECONDARY CONTRIBUTION

The **W3C Baggage propagation pattern** for session correlation in distributed agent systems — applicable beyond the tools evaluated here.

THE CENTRAL TENSION

"OTel compliance" today is a **wire-protocol** claim (OTLP transport), not a **semantic** claim (standard attribute interpretation) — this gap should narrow as OTel GenAI semconv matures.

NEXT STEPS

Standardised agent-level semantic conventions · observability for MCP agent meshes · observability-driven feedback loops

Thank you — questions?

Daniela Otelea
Observability for AI Agents · ZHAW InIT